



TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR



Lab Report No:- 4

Title:- Built in functions in SQL, Group By clause,
Having clause.

Submitted by:-

Name:- surajan shrestha

Roll No:- 38

Submitted To:- Himat chand Thapa

Department Of CSIT

Checked by:-

Date of submission:- 2081/06/04

TITLE: Built in functions in SQL, Group By clause, Having clause

OBJECTIVE:

- To understand the use of SQL built in aggregate functions like `sum()`, `min()`, `avg()`, `count()`.
- To learn how to group data using the Group By clause for summarizing records.
- To explore the HAVING clause, which is used to filter groups created by the Group By clause.
- To execute SQL queries based on various operations such as sorting, aggregating and filtering data efficiently.

THEORY:

→ SQL provides powerful built-in functions & clauses to manipulate and analyze data. Among these, aggregate functions, the Group By clause, and the HAVING clause are essential tools in database management and querying.

→ Aggregate functions:

• It performs calculations on multiple rows of data and return a single value. These functions are often used in combination with Group By to summarize large datasets. Common aggregate functions include:

`sum()`: Returns the total sum of a numeric column.

Eg: `SELECT SUM(salary) FROM Employee;`

`MIN()`: Returns the minimum value from a column.

Eg:

`SELECT MIN(salary) FROM Employee;`

• MAX(): Returns the maximum value from a column.

Eg°

SELECT MAX(price) FROM BOOK;

• AVG(): Returns the average value of a numeric columns.

Eg°

SELECT AVG(price) FROM BOOK;

• COUNT(): Returns the number of rows matching a specific condition.

Eg°

SELECT COUNT(*) FROM Employee;

→ GROUP BY clause°

→ The GROUP BY clause is used in conjunction with aggregate functions to group rows that have the same values in specified columns. It allows you to summarize data and generate reports based on grouped values. For example, grouping employees by department to calculate the total salary per department.

Syntax°

```
SELECT column1, aggregate_function(column2)
FROM table
GROUP BY column1;
```

HAVING clause°

• The ~~the~~ HAVING clause is similar to the WHERE clause but is used to filter results after they have been grouped by the GROUP BY clause. The WHERE clause filters rows before grouping, whereas HAVING filters after the aggregation process.

Syntax :

```
SELECT column1, aggregate_function(column2)
FROM table
```

```
GROUP BY column1
```

```
HAVING aggregate_function(column2) condition;
```

OBSERVATION :

LAB TASKS :

1. Sort the employee records in descending order.
SQL :

```
SELECT * FROM Employee
ORDER BY salary DESC;
```

2. Sort name and publication name in ascending order.

SQL :

```
SELECT * FROM Booklist
ORDER BY name ASC, publication ASC;
```

3. Display top three records from teachers relation
SQL :

```
SELECT TOP 3 * FROM teacher
ORDER BY TID;
```

4. Display the sum of salaries of all employees.
SQL :

```
SELECT sum(salary) AS TotalSalaries
FROM Employee;
```

5. Display the minimum salary of an employee.

SQL:

SELECT MIN(salary) AS minimum salary
FROM Employee;

Display the average price of books written by the same author:

SQL:

SELECT author, AVG(price) AS average price
FROM Book
GROUP BY author;

Display publication name and number of books published by it from the book list relation:

SQL:

SELECT publication, COUNT(*) AS Number of Books
FROM Booklist
GROUP BY publication;

Display the bid and bname of books whose price is greater than the average price of books:

SQL:

SELECT bid, bname
FROM Book
WHERE price > (SELECT AVG(price) FROM Book);

Find the bid, bname and author in ascending order where the author name starts with "S":

SQL:

SELECT bid, bname, author

FROM BOOK

WHERE author LIKE 'S%'

ORDER BY bid, bname, author;

→ find the teacher's name and the book taken by him, where the teacher's salary who takes the book should be the maximum salary:

SQL:

SELECT t.name, b.bname

FROM teacher t

Join Employee e ON t.eid = e.eid

Join Issues i ON t.eid = i.iid

Join BOOK b ON i.bid = b.bid

WHERE e.salary = (SELECT MAX(salary) FROM Employee);

→ find the authors' names who have written more than one book:

SQL:

SELECT author

FROM BOOK

GROUP BY author

HAVING COUNT(*) > 1;

DISCUSSION :

→ In this lab, we explored a variety of SQL functions and clauses essential for analyzing and summarizing data. The use of aggregate functions like SUM(), MIN(), AVG() and COUNT() allowed us to perform

calculations across multiple rows, enabling efficient extraction of insights from large datasets. For instance, calculating the total salary of employees or finding the minimum salary was simplified using these functions. Additionally, grouping data with Group By clause provided an effective way to organize the records by a specific column, such as grouping books by authors to compute the average price of books written by each author & so on.

CONCLUSION:

→ The lab explored the functionality of SQL's built-in functions, Group By, & HAVING clauses. These concepts are crucial for performing calculations and summarizing large datasets efficiently. Understanding these functions allows us to manipulate data in a more structured and meaningful way, making them essential for database management and reporting.